IOT Transmission with Mutual Authentication and Data Integrity

Carter Thomas Powell, Kuan Ruei Chiang

***Abstract*—The Internet of Things (IoT) offers a significant opportunity for individuals to interact with information and objects, leading to an innovative option for everyone to live their lives. Due to their flexibility and limited resources, IoT devices are vulnerable to various security risks such as physical, data modification, and impersonation attacks. Consequently, any protocol designed for IoT systems must take into consideration both security and efficiency in terms of energy consumption, memory requirements, and processing capabilities. To overcome this problem, we introduce a mutual authentication protocol for IoT devices that utilizes verifiable host identification and provides data integrity. It accomplishes all of this while staying small in scale to prevent any overbearing loads on IoT devices. We present this protocol to communicate from IoT devices to the servers, and in turn, from the servers to smartphones. Through a thorough analysis of security and performance, we can present a protocol that is effective against diverse attack types while maintaining a small load in terms of computation, memory usage, and energy consumption.**

I. INTRODUCTION

As time has passed, Internet of Things (IoT) devices have increased dramatically and are being incorporated in just about every aspect of people's lives. Since the term IoT was coined in 1999 there have been countless uses for these small, but powerful devices such as smart lighting, traffic control, air quality control, and anything the mind could imagine that could be connected to the internet. As every aspect of our lives quickly becomes digital with IoT devices, we as a society we've become much more reliant on the internet than any one person would have ever predicted.

This creates new issues in digital security as the rise of IoT devices also brings with it different vulnerabilities to the existing networks and devices. This continuously proves to be a difficult challenge as, until recently, the funding for IoT security has been minimal. It's also proved challenging to implement truly secure protocols as the nature of an IoT device is not similar to any standard computer. This is because the miniscule nature of most of these devices creates limited resources in energy, computing power, memory allocation, and transmission capabilities. With the nature of these devices, applications could include uses in public transportation, law enforcement, power grids, and health care which could all be fatal if not properly protected against malicious attackers. However, with the increase in funding and the increased focus in the area of security, multiple security protocols have been implemented to prevent these malicious attacks.

In this implementation of an IoT transmission, the protocol focuses strictly on IoT devices communicating with pre-identified host servers which then will distribute the data to a user's mobile device. The protocol accomplishes this by utilizing host identification along with non-repeatable hash functions to authenticate both ends of the communication before transmitting the data securely to the preferred destination. Although this protocol provides data integrity and mutual authentication, it cannot be utilized against physical attacks on the IoT device and assumes that the device is in a physically secure location. In the remainder of this paper, it'll explain further into the protocol's design and explain any other assumptions made while developing this lightweight algorithm.

II. NETWORK MODEL AND ASSUMPTIONS

A. Network Model

In this model, the IoT device connects the internet through the protocol presented. This allows the IoT device to securely transmit and store the necessary data for external devices to receive. The devices requesting the information go through a very similar model using our protocol. In this example, the requesting device is a smartphone attempting to access the data stored in the servers by the IoT device. A more visual representation of the process can be seen below in Fig. 1.

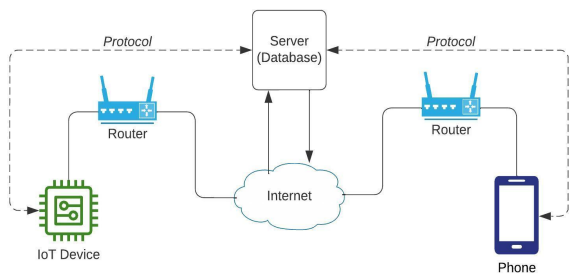

Fig 1. Network Model

B. Assumptions

In this project, we make these assumptions regarding the conditions of the IoT device and the smartphone.

1) The IoT device is located in a physically secure location to prevent any physical malicious attacks
2) The protocol is embedded into the IoT device and preconfigure to accurately communicate to the server
3) The server is a pre-identified entity which is verified and trusted once the mutual authentication is complete
4) While the IoT device is assumed to have minimal resources and processing power the server has enough space to store all information necessary
5) The smartphone can configure the connection at any stage and only can receive data if authorized
6) Along with the server, the smartphone has much more resources than the IoT device.

III. THREAT MODEL

When sending the data from an IoT device to the server, we assume the adversary can intercept the data, send forged data to the server, change miniscule aspects of the data's detail, and resend previous data. On the other side of the transmission, we assume the adversary can falsely identify themselves to send a request to the server to receive data, intercept data being transmitted from the server to the phone, send forged data to the phone pretending to be the server, and repeatedly sending requests to crash the server from the phones connection.

IV. PROPOSED PROTOCOL

The protocol proposed provides mutual authentication along with data integrity for an IoT device when communicating with a server and in turn when the server sends the data to a trusted smartphone. A diagram showing the process in which the protocol functions is shown in Fig. 2. Along with a visual representation of the protocol the steps in which it operates are in this fashion.

1) The IoT device generates a random nonce N_1 and possesses its own ID_A which the server also has. These two aspects are encrypted with a shared key k_{ma} which both the server and IoT device possess and attaches a hashed value of N_1,ID_A, and k_{ma}.This is sent to the server as a request.
2) The server then checks the hashed value for data integrity and authenticates the IoT device with its ID_A and encryption using k_{ma} . If everything passes it then generates its own nonce N_2. This is used to send the IoT device back N_2 , N_1 , and ID_A encrypted with k_{ma} and the hashed value of N_2 , N_1 , ID_A , and k_{ma}.
3) The IoT device receives this confirmation from the server and can authenticate the server and check its integrity using the received hash value. If everything passes the IoT device sends the N_2 along with the data k_s encrypted with the shared key k_{ma} and the hashed value of N_2 , ID_A , and k_{ma}.

This terminates the connection and requirements between the IoT device and the server. The following steps show the process of the phone receiving the IoT data from the server.

4) The phone generates a random nonce N_3 and possesses its own ID_B which the server also has. These two aspects are encrypted with a shared key k_{mb} which both the server and phone has and attaches a hashed value of N_3 , ID_B , and k_{mb}.This is sent to the server as a request.
5) The server then checks the hashed value for data integrity and authenticates the phone with its ID_B and encryption using k_{mb} . If everything passes it then generates its own nonce N_4. This is used to send the phone back N_4 , N_3, and ID_B encrypted with k_{mb} and the hashed value of N_4 , N_3, ID_B, and k_{mb}.
6) The phone receives this confirmation from the server and can authenticate the server and check its integrity using the received hash value. If everything passes the phone generates a nonce N_5 and sends N_5, N_4 , ID_B encrypted with the shared key k_{mb} along with the hashed value of N_5 , N_4 , ID_B , and k_{mb}.
7) If the hash from the phone passes, mutual authentication is valid from the phone and the server.

The server then can send the IoT data to the phone by encrypting N_5 and k_s using k_{mb} and appending the hashed value of N_5 , k_s and k_{mb}.

This creates a complete transmission of data that provides data integrity and mutual authentication from an IoT device to a server and in relation to an authorized smartphone.

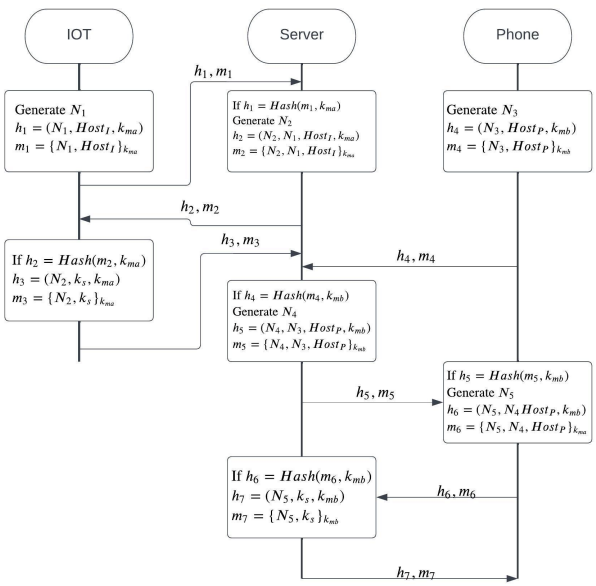

Fig. 2 Mutual authentication and data integrity between an IoT device, a server, and a smartphone

V. SECURITY ANALYSIS

In order to create a secure environment, we use mutual authentication.

1) There is mutual authentication between the IoT device and the server as well as the phone and the server.
2) We have a pre-shared key between IoT and the server as well as between the phone and the server.

A. Protection against data modification attacks and impersonation attacks

Two of the security requirements when sending data is to protect the data from getting obtained and modified by the adversary as well as making sure the server knows who it's talking to. It provides data integrity by combining nonces with the message and concatenating hashed values with it. In our program, the server will also check the identity of the IoT device and the phone before sending or receiving data. It accomplishes this by encrypting messages with a pre-shared key that only the server and the host have along with including the host identification number in the messages and hashes.

B. Protection against replay attack

Without timestamps it's difficult to prevent replay attacks with the protocol. One solution the protocol provides is the new nonce generation in each iteration of data sent. This allows the database to not be overloaded with the same message after it's been sent once.

VI. AUTOMATED SECURITY VERIFICATION

In this section, we will show our protocol implementation in ProVerif.

1) *Completeness*: The server will provide any bitstring input or output when the device passes the checks.
2) *Deadlock Freeness*: There are no aspects of the protocol which cause deadlocks.
3) *Livelock or Tempo-Blocking Freness*: There are no infinite loops in the program that keep it running.
4) *Termination*: After the protocol starts, it will either accept the input or output, or decline authorization.
5) *Absence of Nonexecutable Interactions*: The protocol will only receive inputs, and give outputs under normal operating conditions.

We utilize host identifications in our program to verify the identity of the IoT device and the phone. When either of them attempts to communicate with the server, they will need to send a request containing a hash message and a message encrypted by their pre-shared key. The IoT device and the server have a pre-shared key as well as the phone and the server have another pre-shared key. Both messages will also have a host identification with them to confirm their identity. After the server checks the identity, it will send a confirmation to them. When IoT receives the confirmation, it will send the data to the server and the server will store it. When the phone receives the confirmation, it will check the message it receives from the server and send another confirmation back to the server. When the server receives the message and confirms it, it will send the information from the IoT device that has been stored in the database to the phone as requested.

In Fig. 3 we can see the results of the protocol after executing it inside of ProVerif. From this output, we can see that in fact, this protocol contains both data integrity and

confidentiality. We can also see that the adversary cannot obtain the message k_s.

```
Verification summary:

Query not attacker(k_s[]) is true.

Query not attacker(test[]) is false.

Query inj-event(serverTerms1(m,k)) ==> inj-event(IOTAccepts(m,k)) is true.

Query inj-event(IOTTerms(m,k)) ==> inj-event(serverAccepts(m,k)) is true.

Query inj-event(phoneTerms(m,k)) ==> inj-event(serverAccepts(m,k)) is true.

Query inj-event(serverTerms2(m,k)) ==> inj-event(phoneAccepts(m,k)) is true.
```

Fig. 3 ProVerif output for the proposed protocol

VII. PERFORMANCE ANALYSIS

When analyzing the complexity of this protocol, there are various steps we first need to take. The first of which includes the transmission from the IoT device and the server. In this proposal the IoT creates one nonce, has two encryptions, one decryption, and three hash functions. The server in return to the IoT device generates one nonce, has one encryption, two decryptions, and three hash functions. The second step involves the server's transmission to the phone. In this proposal the phone creates two nonces, has two encryptions, two decryptions, and four hash functions. The server in return creates one nonce, two encryptions, two decryptions, and four hash functions. These are the main computational aspects of the protocol and can be visualized in Fig. 4 below. In this figure N_H is the number of hash functions, N_N is the number of nonces generated, N_{ENC} is the number of encryptions, and N_{DEC} is the number of decryptions present for each of the three entities in one full transmission.

IoT Device	Server	Phone
$N_N + 2N_{ENC} + N_{DEC} + 3N_H$	$2N_N + 3N_{ENC} + 4N_{DEC} + 7N_H$	$2N_N + 2N_{ENC} + 2N_{DEC} + 4N_H$

Fig. 4 A table showing the functions required by each entity

Going into further detail of the computational complexity of this protocol, we first need to analyze the complexity of each of these functions. The nonce generation consists of forming random bitstrings and in turn can be considered $O(n)$. Both the encryption and decryption functions are also linear and can be considered $O(n)$. Lastly, the hashing function goes through the predefined message and converts it to a specific bitstring. Since it converts through the message one character at a time it is also considered linear, or $O(n)$. With all four of these functions being linear, we can consider the entire protocol as running a $O(n)$ computational complexity.

VIII. CONCLUSION

The protocol proposed provides mutual authentication and data integrity between IoT devices, a trusted predefined server, and a smartphone. This protocol addresses the restraints presented by the lack of resources IoT devices possess along with working with devices that have much more processing and memory capabilities. It accomplishes this by pre-sharing secret keys between the IoT device and the server as well as the server and the smartphone. Other methods the protocol uses to provide mutual authentication and data integrity include host identification numbers, hash functions, and nonce generations. All of these combined provide ample digital security to the data being transmitted by the IoT device without using the resources that standard computers provide on a daily basis. It's still best to utilize this protocol in select scenarios as physical security is not provided by this protocol and the multiple transmissions required to provide mutual authentication may not provide a feasible solution for applications requiring time sensitivity. Nonetheless, this solution can provide ample security for a number of different applications and provides another option for keeping IoT data secure.

IX. REFERENCES

[1] M. N. Aman, K. C. Chua, and B. Sikdar, Mutual authentication in IOT systems using physical ... - IEEE xplore, https://ieeexplore.ieee.org/document/7924368 (accessed May 8, 2023).

[2] IEEE Reference Guide - IEEE Author Center, https://ieeeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf (accessed May 8, 2023).

[3]K. D. Foote. "A Brief History of the Internet of Things - DATAVERSITY." Dataversity. Jan. 14, 2022. https://www.dataversity.net/brief-history-internet-things/ (accessed: May 9, 2023).